

第 1 课 识别颜色并与 arduino 串口通信夹取木块

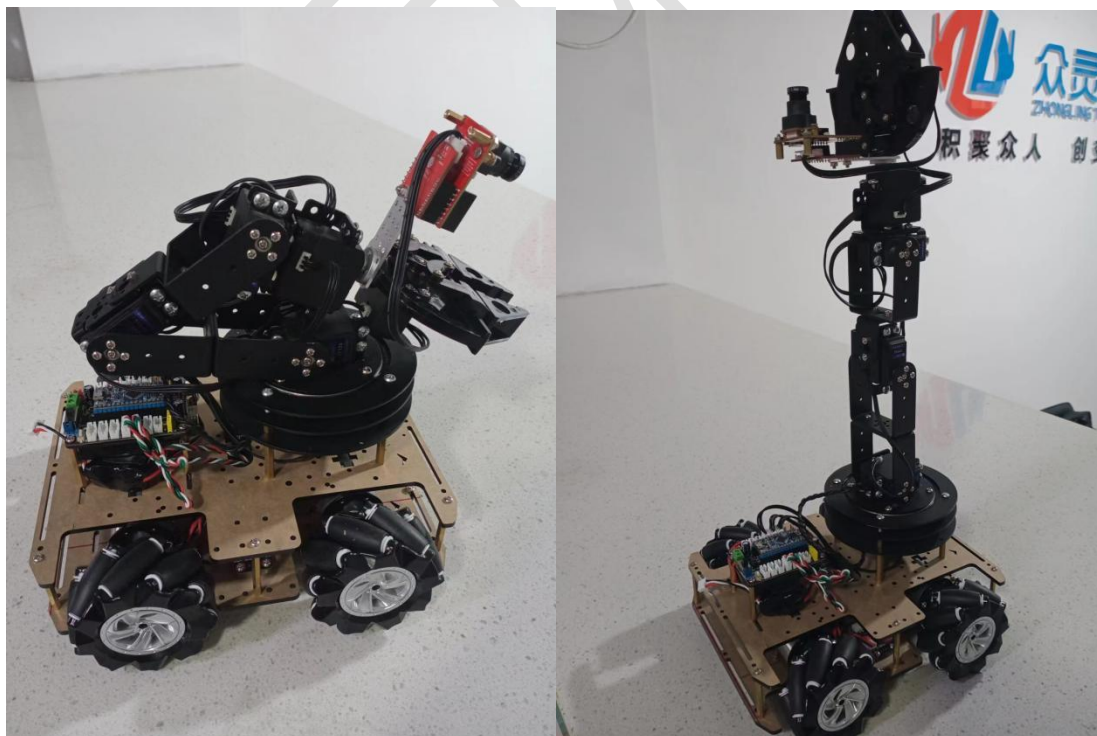
【课程目标】：

本课学习使用 openmv 识别颜色并通过串口发送数据到 arduino 中来让机械臂夹取对应颜色的木块到指定位置。

注意：本节主要是演示 openmv 与 arduino 间的串口通信功能，硬件方面使用的是我们店里的产品，如果大家购买了我们对应的产品，也可以实际操作一下的，如果没有购买的话，也可以根据代码和下面的讲解来深入了解 openmv 与 arduino 之间的串口通信，其他控制板的话也是类似的用法。

【课前准备】：

Openmv、C5 轮式底盘车+总线机械臂、三合一底板+arduino 核心板、usb 数据线



【本课内容】：

一、代码

1.openmv 端代码

```

1.  ...
2.  颜色识别夹取木块
3.  红色木块放在正前方，绿色木块放在小车左侧，蓝色木块放在小车右侧
4.  红绿蓝三种颜色可以同时识别，还可自行添加其他颜色阈值来识别其他颜色
5.  # 注意：环境光会对识别有影响，找比较合适的环境来运行此程序。
6.  颜色阈值采集方法：
7.  方法一：
8.      1.可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集，
9.      2.拖动 6 个滑块来调整图像，直到要采集的颜色都呈白色，其他都呈黑色为止，
10.     3.采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
11. 方法二：
12.     1.可以在右侧‘帧缓冲区’和‘直方图’的‘LAB 色彩空间’配合采集
13.     2.先在帧缓冲区中用鼠标左键框出要采集的颜色区域，
14.     3.然后在直方图中的 LAB 色彩空间中，分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
15.  ...
16. import sensor, time
17. from pyb import UART
18. import utime
19. import json
20.
21.
22. target_color_appear_count = 0
23.
24. # 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
25. thresholds = {'red': (62, 41, -9, 113, 24, 98),          # 红色阈值
26.               'green': (68, 26, -57, -18, 100, -81),    # 绿色阈值
27.               'blue': (70, 6, -76, 36, -20, -96)        # 蓝色阈值
28.               }
29. # 各颜色对应要执行的动作组
30. actions = {'start': '$DGT:3-3,1!',
31.            'red': '$DGT:4-10,1!',
32.            'green': '$DGT:11-19,1!',
33.            'blue': '$DGT:20-28,1!'
34.            }
35.
36. # 初始化函数
37. def init_setup():
38.     global sensor, clock, uart3          # 设置为全局变量
39.     sensor.reset()                       # 初始化感光元件
40.     sensor.set_pixformat(sensor.RGB565)  # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图，
                                           GRAYSCALE 为灰度图)

```

```

41.     sensor.set_framesize(sensor.QVGA)           # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
42.     sensor.skip_frames(10)                      # 跳过一些帧，使以上的设置生效
43.     sensor.set_auto_gain(False)                 # 关闭自动增益。在颜色识别中，需要关闭自动增益
44.     sensor.set_auto_whitebal(False)             # 关闭白平衡。白平衡是默认开启的，在颜色识别中，需要关闭白平
衡
45.     clock = time.clock()                        # 创建时钟对象
46.     uart3 = UART(3, 115200)                    # 初始化串口 3，设置波特率为 115200
47.     print('初始化完成')
48.     utime.sleep(2)
49.     uart3_send(actions['start'])
50.     utime.sleep(2)
51.
52.
53.     # 串口接收数据
54.     def uart3_recv():
55.         recv_data = uart3.read()
56.         print('recv data: %s' % recv_data)
57.         return recv_data
58.
59.     # 串口发送数据
60.     def uart3_send(send_data):
61.         global uart3
62.         uart3.write(send_data)
63.
64.
65.     # 查找图像中最大的色块
66.     def find_max(blobs):
67.         max_size = 0
68.         for blob in blobs:
69.             if blob[2] * blob[3] > max_size:
70.                 max_blob = blob
71.                 max_size = blob[2] * blob[3]
72.         return max_blob
73.
74.     # 主函数
75.     def main():
76.         global target_color_appear_count
77.         while 1:                                # 无限循环
78.             clock.tick()                        # 更新 FPS 帧率时钟
79.             img = sensor.snapshot()              # 拍一张照片并返回图像
80.             # 循环 3 种颜色阈值，在图像中查看是否有对应颜色并做一些操作，thresholds 需为字典格式
81.             for threshold in thresholds:
82.                 blobs = img.find_blobs([thresholds[threshold]])          # 在图像中查找目标颜色区
域

```

```

83.         if blobs:                                     # 当查找到目标颜色区域
            后
84.             max_blob = find_max(blobs)                 # 查找图像中最大的目标色
            块
85.             img.draw_rectangle(max_blob[0:4])           # 将找到的目标颜色区域用
            矩形框出来
86.             img.draw_cross(max_blob[5], max_blob[6])    # 在目标颜色区域的中心点
            处画十字
87.             target_area = max_blob[2] * max_blob[3]     # 面积
88.             print('area is ', target_area)              # 打印面积
89.
90.             if 2000 < target_area < 6000:                # 面积合适时
91.                 target_color_appear_count += 1
92.
93.             if target_color_appear_count > 5:            # 识别到的面积合适时的次
            数大于 5 次时串口发数据
94.                 if threshold == 'red':
95.                     uart3_send(actions['red'])           # 串口将识别到的颜色对应
            的动作组发出去
96.                     utime.sleep(15)                     # 延时以便让动作组执行完
            成
97.                 elif threshold == 'green':
98.                     uart3_send(actions['green'])         # 串口将识别到的颜色对应
            的动作组发出去
99.                     utime.sleep(18)                     # 延时以便让动作组执行完
            成
100.                elif threshold == 'blue':
101.                    uart3_send(actions['blue'])          # 串口将识别到的颜色对应
            的动作组发出去
102.                    utime.sleep(18)                     # 延时以便让动作组执行完
            成
103.                    print('uart3 send color is ', threshold)
104.                    target_color_appear_count = 0
105.
106.
107. # 程序入口
108. if __name__ == '__main__':
109.     init_setup()                                         # 执行初始化函数
110.     try:                                                 # 异常处理
111.         main()                                           # 执行主函数
112.     except:
113.         pass

```

2.arduino 端代码可参考资料中的 ‘Carbot(无 C6)-智能车出厂程序 20210925’

二、代码使用方法

第一步：将 openmv 的代码保存到 openmv 中

第二步：将 arduino 端代码上传到 arduino 中

第三步：下载动作组到 arduino 中

第四步：将 openmv 用一根总线接到舵机上，打开控制板上的电源开关，等待 2 秒后机械臂会先弯下去，然后将红色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车正前方；将绿色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车左侧；将蓝色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车右侧。

三、运行效果

将 openmv 用一根总线接到舵机上，打开控制板上的电源开关，等待 2 秒后机械臂会先弯下去，然后将红色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车正前方；将绿色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车左侧；将蓝色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车右侧。

注意：如果颜色识别的有问题，可以根据前面的教程修改 openmv 端代码中的对应颜色阈值即可。使用时，摄像头画面中尽量不要有干扰的颜色，手有可能被识别为红色，大家要注意一下。

四、知识点

关于代码中 UART 的使用可以参考官方文档：

<https://docs.singtown.com/micropython/zh/latest/openmvcam/library/pyb.UART.html#pyb-uart>

【本课小结】：

本课学习了使用 openmv 识别颜色并通过串口发送数据到 arduino 中来让机械臂夹取对应颜色的木块到指定位置。

注意使用时摄像头画面中不要有干扰的颜色。

注意：这个 openmv 示例代码同样适用于我们的多合一 32 控制板和三合一 51 控制板，只需上传轮式车的代码到 32 或 51 中就可以了。